

Names: Rhilo Sotto, John Hayali, Eren Ugur

Report for Programming Assignment #4

Explanation of Simulation Code

Representation of Page Tables

The simulation has each process having its own page table, which is in the form of a single-level page table with 128 entries. The structure is based on the 16-bit virtual address space and a page size of 512 bytes, producing 7 bits of the virtual page number. One virtual page is represented by a page table entry.

A page table entry contains each page translation field, which holds the number of the physical page frame in the event the page is already loaded in memory, or is blank otherwise. Each entry also has a dirty bit to show whether the page has been written to, a referenced bit to show whether the page has been accessed recently, and an auxiliary field containing the metadata required by algorithms such as timestamps or an index into the future.

Every process has an independent page table, and so address translation and page replacement decisions are independent for each process, with no shared memory between processes.

Virtual-to-Physical Address Translation

Each memory reference is decoded in the simulator by right shifting the virtual address by nine bits to retrieve the virtual number of the page. This operation divides the 7 largest bits which represent the virtual page number and the 9 smallest bits which represent the offset in the page.

The virtual page number is then indexed in the page table of the respective process. In case the entry in the page table has a valid physical frame number, the access is considered a memory hit. Though the final physical address of the byte is not explicitly computed by the simulator, it is conceptually constructed as the sum of the physical frame number and the page offset. The simulation focuses on page-level behavior rather than byte-level address translation.

Determining When There Is a Need to Replace a Page

When a virtual page is accessed through a memory reference, a page replacement will be needed in case such a page is not available in physical memory. This is identified by an empty translation field of the entry in the page table. Once this happens, a page fault is created, and the page replacement algorithm is called.

Once the page fault has been processed and the page is loaded into the memory, the simulator updates the page metadata. In case the access type is a write, the dirty bit is set. Each access to memory sets the bit referenced to indicate a recent usage. The simulator used with the PER algorithm would periodically clear all the referenced bits every 200 memory accesses.

Determining the Replaced Page

Physical memory of the model is expressed as a 32-page frame size array. The frames are either page table entries or empty. The simulator will first check whether there is a free frame available when there is a page fault.

In case there is a free frame, the page is loaded, without replacing an existing page. When the physical memory is at maximum capacity, the simulator picks one of the victim pages according to the active replacement algorithm. The RAND algorithm is used to choose a random frame, using the Python random library and a seed number that changes depending on the trial. FIFO overwrites the oldest page in the memory. The least recently accessed page is replaced with LRU. PER gives priority to unreferenced and clean pages, unreferenced dirty pages, referenced clean pages, and referenced dirty pages.

Explanation of Our Own Replacement Algorithm (MIN)

In the case of the MIN algorithm, the simulator involves an oracle-based approach with full knowledge of the future memory access trace. The entire trace is read into memory before simulation begins. After each access, the simulator scans forward in the trace to determine the next time the same process accesses the same virtual page. This future access index is stored in the page's auxiliary field.

When a page fault occurs, and physical memory is full, the simulator evicts the page whose next access occurs farthest in the future. This is implemented by selecting the page with the largest auxiliary value, with ties broken by choosing the lowest physical frame index. If the selected victim page is dirty, an additional disk write is recorded before the page is replaced.

The MIN algorithm guarantees the minimum possible number of page faults for a given trace, but it has significantly higher runtime than the other algorithms due to the required future trace analysis.

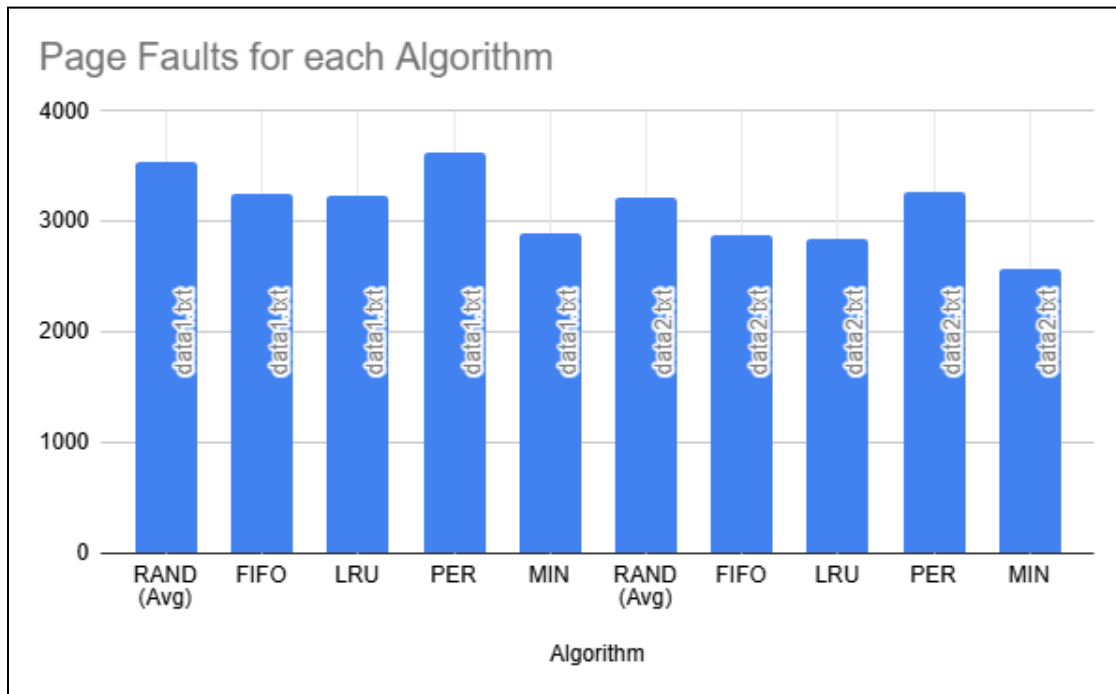
Results

Results:  COMPE 571 PA4 Data

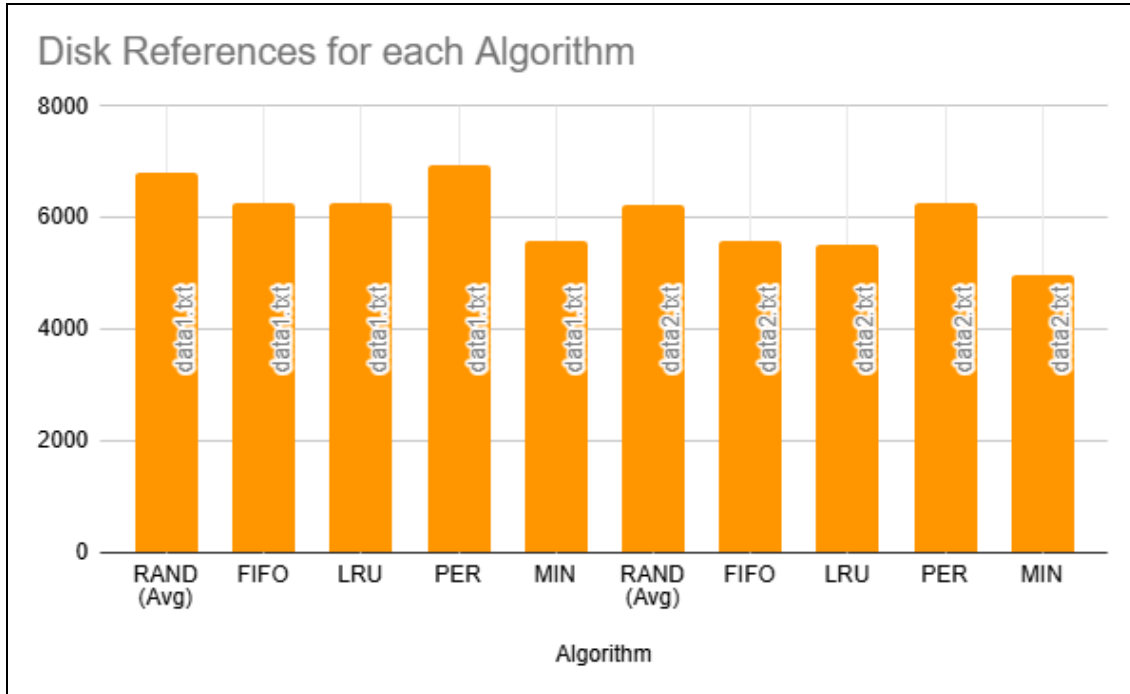
The following table displays the statistics for each replacement algorithm:

Algorithm	Input File	Page Faults	Disk References	Dirty Writes
RAND (Avg)	data1.txt	3542	6812	3270
FIFO	data1.txt	3248	6278	3030
LRU	data1.txt	3239	6248	3009
PER	data1.txt	3632	6928	3296
MIN	data1.txt	2891	5575	2684
RAND (Avg)	data2.txt	3221	6224	3003
FIFO	data2.txt	2878	5595	2717
LRU	data2.txt	2844	5503	2659
PER	data2.txt	3272	6278	3006
MIN	data2.txt	2570	4971	2401

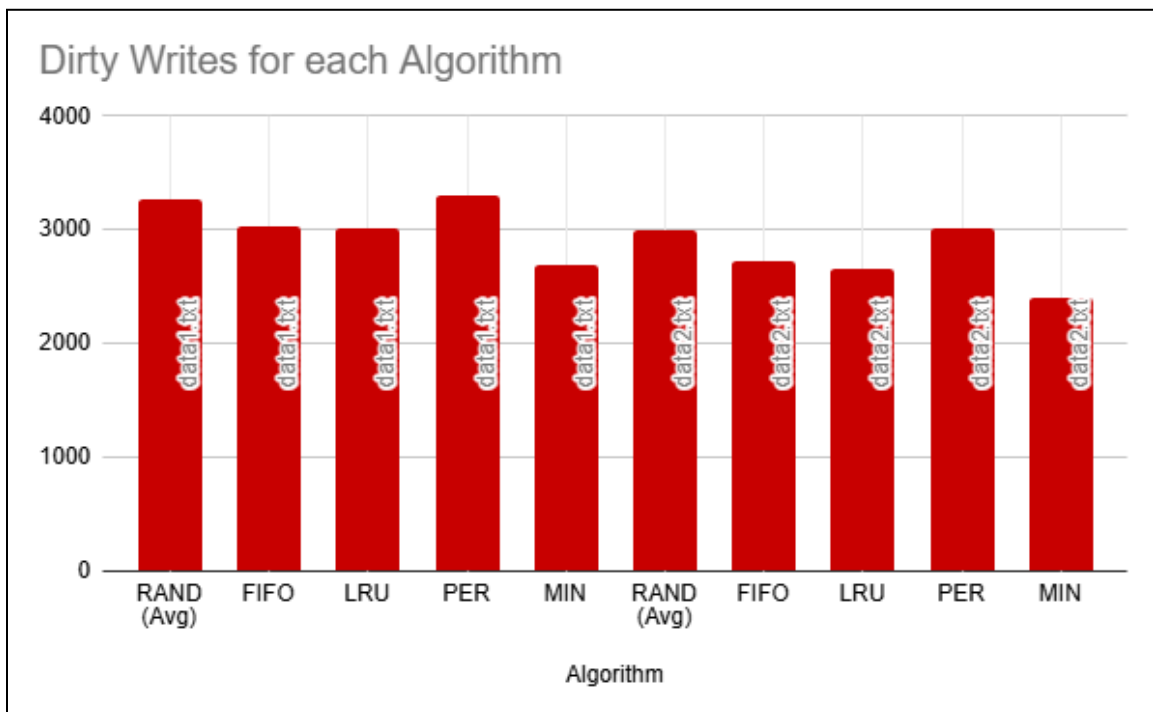
Moreover, the following three graphs compare the total number of page faults, disk accesses, and dirty page writes between each replacement algorithm:



Graph of Page Faults for each Algorithm



Graph of Disk References for each Algorithm



Graph of Dirty Writes for each Algorithm

It appears that our MIN algorithm outperforms the rest of the algorithms across the total number of page faults, disk references, and dirty writes. Aside from MIN, FIFO and LRU are very comparable, with LRU slightly beating out FIFO for both data sets. PER was surprisingly the worst performing algorithm, having the highest values for all three statistics (page faults, disk references, and dirty writes). This is likely due to the fact that PER loses memory history after every 200 memory references. Thus, when all of the reference bits are set to 0, the algorithm treats a heavily-used page the same as an unused page. As a result, a page that was recently used many times can be chosen as the victim page, which is a poor choice because the next memory access would likely lead to a page fault.